Android Setup Guide

Expo CRUD Starter: from a fresh Mac to the app running on an Android emulator

Written for people with no previous React Native or Android experience. Follow the parts in order. Each grey box is a command you can copy into **Terminal** (press `Cmd + Space`, type *Terminal*, press Enter).

Platform: macOS (Apple Silicon or Intel). **Time:** about 1–2 hours the first time, mostly downloads.

Contents

1. Why we need Android Studio (native modules)
2. What you need before starting
3. System setup: install the tools
4. Android Studio: first-time setup wizard
5. Android Studio: SDK Manager (what to tick)
6. Environment variables (JAVA_HOME, ANDROID_HOME)
7. Create an emulator (virtual phone)
8. ADB: connecting emulators and real phones
9. Run the application
10. Daily workflow
11. Troubleshooting
12. Quick checklist

1. Why we need Android Studio (native modules)

Many Expo tutorials say "install the **Expo Go** app on your phone and scan a QR code". **That does not work for this project.**

What is a native module?

A React Native app has two layers:

- **JavaScript/TypeScript layer:** our screens, logic and styling (everything in `src/`).
- **Native layer:** real Android (Kotlin/Java/C++) code compiled into the app. Libraries that need phone features or very high speed ship their own native code. These are called **native modules**.

This project uses several libraries with native code, for example:

Library	What it does in our app
`react-native-mmkv` + `react-native-nitro-modules`	Fast, encrypted local storage (login token, theme, language)
`react-native-reanimated` + `react-native-worklets`	Smooth animations
`react-native-keyboard-controller`	Keyboard handling on forms
`react-native-gesture-handler`, `@gorhom/bottom-sheet`	Swipes and bottom sheets
`react-native-restart`	Restarts the app after changing language
`expo-dev-client`	Turns our app into a "development build" with a developer menu

Why Expo Go is not enough

Expo Go is a ready-made app from the store. It only contains a **fixed set** of native modules chosen by Expo. Libraries like **MMKV / Nitro Modules** and **react-native-restart** are not inside it. If you open our project in Expo Go, it crashes with errors such as:

```
Error: NitroModules are not supported in Expo Go!
Invariant Violation: TurboModuleRegistry.getEnforcing(...): 'xyz' could not be found
```

The solution: a development build

We compile **our own** version of the app on our computer. It contains every native module we use. This is called a **development build**. Compiling an Android app needs:

- **JDK 17:** Java, used by the Android build tool (Gradle)
- **Android SDK:** Android libraries and build tools
- **An emulator or a real phone** to install the app on

Android Studio is the easiest way to install the Android SDK and the emulator. We don't write code in Android Studio. We only use it for setup and to start the emulator.

```
pnpm android
    │
    ├─ 1. Expo generates the  android/  folder (native project)
    ├─ 2. Gradle + JDK 17 + Android SDK compile it  →  app-debug.apk   (slow, first time 5–15 min)
    ├─ 3. ADB installs the APK on the emulator / phone
    └─ 4. Metro bundler serves our JavaScript  →  app opens          (fast, every time)

Later, JS/TS changes  →  only step 4 (instant Fast Refresh)
New native library or app.config.ts change  →  run  pnpm android  again
```

2. What you need before starting

Item	Requirement
Computer	Mac with macOS 13 or newer (Apple Silicon M1/M2/M3/M4 recommended)
Memory (RAM)	8 GB minimum, **16 GB recommended** (the emulator uses a lot)
Free disk space	**At least 30 GB** (Android SDK + emulator image + project)
Internet	Stable connection. Several GB of downloads the first time.
Access	Your Mac admin password and access to the GitHub repository
Expo account	**Not needed** to run the app locally

To check whether your Mac is Apple Silicon or Intel, click **Apple menu** → **About This Mac**. **Chip: Apple M…** means Apple Silicon. **Processor: Intel…** means Intel.

3. System setup: install the tools

3.1 Install Homebrew (the Mac package installer)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

When it asks for your password, type your Mac login password and press Enter. **Nothing appears while you type. This is normal.**

At the end, Homebrew prints **"Next steps"**. Run those lines. On Apple Silicon they are:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Check:

```
brew -v
```

3.2 Install Node.js, Watchman, Git, Java 17 and Android Studio

```
brew install node watchman git
brew install --cask zulu@17 android-studio
```

- **node:** runs JavaScript tools (version 20 or newer is required)
- **watchman:** watches files so the app reloads when you save
- **zulu@17:** Java Development Kit 17, needed by Android builds. **Use 17, not newer.**
- **android-studio:** installs the Android Studio app into Applications

3.3 Install pnpm (our package manager)

```
npm install -g pnpm
```

> **Important:** this project **only** works with `pnpm`. `npm install` and `yarn` are blocked on purpose.

3.4 Check the versions

```
node -v
pnpm -v
java -version
git --version
```

Expected: Node **v20** or higher, pnpm **10.x**, Java **"17.x"**. If `java -version` shows a different version, finish Part 6 first, then check again.

4. Android Studio: first-time setup wizard

Open **Applications** → **Android Studio**. If macOS asks "*Are you sure you want to open it?*", click **Open**.

Screen you see	What to select
Import Android Studio Settings	**Do not import settings** → OK
Help improve Android Studio (data sharing)	Your choice: **Don't send** is fine
Welcome	**Next**
Install Type	**Standard** → Next
Select UI Theme	Any (Darcula / Light) → Next
Verify Settings	Check that it lists **Android SDK**, **Android SDK Platform**, **Android Emulator** and **Performance (Android Emulator hypervisor)** → Next
License Agreement	Click **each** item on the left (e.g. *android-sdk-license*, *android-sdk-preview-license*) and choose **Accept** for every one → Finish
Downloading Components	Wait. This can take 10–30 minutes. Then click **Finish**.

> The SDK is installed in `~/Library/Android/sdk` (i.e. `/Users/<your-name>/Library/Android/sdk`). We'll use this path in Part 6.

> If the wizard does not appear, or you skipped it by mistake, open Android Studio and go to **More Actions** → **SDK Manager** and follow Part 5.

5. Android Studio: SDK Manager (what to tick)

On the Android Studio welcome screen, click **More Actions** → **SDK Manager**. If a project is open, use **Android Studio** → **Settings** → **Languages & Frameworks** → **Android SDK** instead.

Tab 1: SDK Platforms

Tick **Show Package Details** (bottom right), then expand **Android 16.0 ("Baklava") – API Level 36** and tick:

- ☑ **Android SDK Platform 36**
- ☑ **Google Play ARM 64 v8a System Image** (Apple Silicon Macs)
  or ☑ **Google Play Intel x86_64 Atom System Image** (Intel Macs)

Expo SDK 54 builds against API 36. If you need to test older Android versions, you can install additional system images later.

Tab 2: SDK Tools

Tick **Show Package Details** and make sure these are ticked:

- ☑ **Android SDK Build-Tools** → expand and tick **36.0.0**
- ☑ **Android SDK Command-line Tools (latest)**
- ☑ **Android Emulator**
- ☑ **Android SDK Platform-Tools** (this contains `adb`)

- □ NDK (Side by side) and CMake are **optional**. The build downloads the right versions automatically when needed.

Click **Apply** → **OK**, accept the licenses, and wait for the download to finish.

6. Environment variables (JAVA_HOME, ANDROID_HOME)

Terminal needs to know where Java and the Android SDK are. Copy this block **exactly once**:

```
cat >> ~/.zshrc <<'EOF'

# ---- React Native / Android ----
export JAVA_HOME=/Library/Java/JavaVirtualMachines/zulu-17.jdk/Contents/Home
export ANDROID_HOME=$HOME/Library/Android/sdk
export PATH=$PATH:$ANDROID_HOME/emulator:$ANDROID_HOME/platform-tools
EOF
source ~/.zshrc
```

Now **close Terminal completely (Cmd + Q)** and open it again. Then check:

```
echo $JAVA_HOME
echo $ANDROID_HOME
java -version
adb --version
emulator -version
```

All five commands should print something. None of them should say `command not found`.

> If `echo $JAVA_HOME` works but `java -version` shows the wrong version, list your installed JDKs with `/usr/libexec/java_home -V` and make sure `zulu-17.jdk` is in the list.

7. Create an emulator (virtual phone)

1. Open Android Studio. On the welcome screen click **More Actions** → **Virtual Device Manager**.
2. Click **+ (Create Virtual Device)**.
3. **Select Hardware:** category **Phone** → choose **Pixel 8** (any recent Pixel is fine). Prefer devices with the **Play Store** icon → **Next**.
4. **System Image:** choose **API 36** (*Baklava* / *Android 16*), **Google Play**, with ABI:
   - **arm64-v8a** on Apple Silicon Macs
   - **x86_64** on Intel Macs

   If a download icon ⬇ appears next to it, click it and wait. Then click **Next**.
5. **Verify Configuration:**
   - AVD Name: keep the default, e.g. `Pixel_8` (no spaces is easiest)
   - Startup orientation: Portrait
   - Click **Show Advanced Settings** (in newer versions this is the *Additional settings* tab):
     - Graphics: **Hardware** (or *Automatic*)
     - RAM: **4096 MB** if your Mac has 16 GB, otherwise **2048 MB**
     - Internal Storage: **8 GB** or more (our debug app is large)

   Click **Finish**.
6. In the Device Manager list, click ▶ **(Play)** next to your device. A phone window opens and boots Android. The first boot can take 1–3 minutes.

Start the emulator from Terminal (optional, faster)

```
emulator -list-avds          # shows your emulator names, e.g. Pixel_8
emulator -avd Pixel_8        # starts it (keep this Terminal tab open)
```

If the emulator acts strangely, use a cold boot: `emulator -avd Pixel_8 -no-snapshot-load`

8. ADB: connecting emulators and real phones

ADB (Android Debug Bridge) is the tool that lets your Mac talk to an Android device. It installs the app and forwards network ports. It comes with *Android SDK Platform-Tools*.

8.1 Check connected devices

With the emulator running:

```
adb devices
```

Expected output:

```
List of devices attached
emulator-5554   device
```

Status shown	Meaning	Fix
`device`	Connected and ready ✅	Nothing to fix
(empty list)	No device found	Start the emulator, or plug in your phone and enable USB debugging (8.3)
`offline`	Device is still booting or stuck	Wait 30 seconds, or run the restart commands in 8.2
`unauthorized`	Phone hasn't trusted this Mac	Unlock the phone and tap **Allow** on the "Allow USB debugging?" popup

8.2 Restart ADB (fixes most connection problems)

```
adb kill-server
adb start-server
adb devices
```

8.3 Use a real Android phone instead of an emulator (USB)

1. On the phone, open **Settings** → **About phone** and tap **Build number 7 times**. You'll see "*You are now a developer*".
2. Go to **Settings** → **System** → **Developer options** and turn on **USB debugging**.
3. Connect the phone to the Mac with a **data** USB cable. Some cables only charge.
4. On the phone, tap **Allow** (tick *Always allow from this computer*).
5. Run `adb devices`. Your phone's serial number should appear with status `device`.
6. Let the phone reach the local mock API through the USB cable:

   ```
   adb reverse tcp:3001 tcp:3001     # mock API
   adb reverse tcp:8081 tcp:8081     # Metro bundler (Expo usually does this automatically)
   ```

   With `adb reverse`, the phone can use `http://localhost:3001/` in `.env`. Run these again each time you reconnect the cable.

8.4 Wireless debugging (Android 11+, optional)

Phone and Mac must be on the same Wi-Fi. On the phone, open **Developer options** → **Wireless debugging** → **Pair device with pairing code**. Then on the Mac:

```
adb pair <ip>:<pairing-port>      # enter the 6-digit code shown on the phone
adb connect <ip>:<port>           # the IP:port shown on the Wireless debugging screen
adb devices
```

8.5 Several devices connected?

If `adb devices` lists more than one device, `pnpm android` asks which one to use. You can also pick one directly:

```
pnpm android --device
```

9. Run the application

9.1 Get the code

```
cd ~/Desktop
git clone https://github.com/saiphoenix/expo-crud-starter.git
cd expo-crud-starter
```

If GitHub asks you to log in, ask the repository owner to add you as a collaborator first.

9.2 Install dependencies (the first command after cloning)

```
pnpm install
```

This downloads all libraries into `node_modules/` and takes a few minutes.

9.3 Set the API address in `.env`

The project includes a `.env` file that points to a local fake backend (the **mock API**). An Android emulator can't reach your Mac at `localhost`. It uses the special address **10.0.2.2** instead.

Where the app runs	`EXPO_PUBLIC_API_URL` in `.env`
Android **emulator**	`http://10.0.2.2:3001/`
Real phone over USB with `adb reverse` (8.3)	`http://localhost:3001/`
Real phone over Wi-Fi	`http://<your-Mac-IP>:3001/` (find the IP with `ipconfig getifaddr en0`)
Remote demo API (no local backend)	`https://dummyjson.com/`

For the emulator, run:

```
sed -i '' 's#localhost:3001#10.0.2.2:3001#' .env
cat .env
```

You should see `EXPO_PUBLIC_API_URL=http://10.0.2.2:3001/`.

9.4 Start the mock API (Terminal tab 1, keep it open)

```
pnpm mock:api
```

Expected: 🚀 `Mock API on http://localhost:3001`. Every product you create, edit or delete is saved in `mock-api/db.json`.

9.5 Start the emulator

Start it from Android Studio (Part 7, step 6) or with `emulator -avd Pixel_8`. Confirm with `adb devices`.

9.6 Build and run the app (Terminal tab 2)

Press `Cmd + T` to open a new Terminal tab, then run:

```
cd ~/Desktop/expo-crud-starter
pnpm android
```

What happens:

1. Expo creates the `android/` folder. This is generated code; never edit it by hand.
2. Gradle compiles the native app. **The first time takes 5–15 minutes** and downloads a lot. Later builds are much faster.
3. The app is installed on the emulator and opens automatically.
4. Metro (the JavaScript bundler) keeps running in this tab. Leave it open.

9.7 Use the app

1. Onboarding screen → tap **Let's Get Started**
2. Login → this is a demo login. Use **any email** and **any 6-character password**, e.g. `test@test.com` / `123456`
3. Tabs: **Feed** (template example), **Products** (our CRUD reference), **Style** (UI kit), **Settings**

> **Done!** Change any text in `src/` and save. The app updates instantly (Fast Refresh).

10. Daily workflow (after the first build)

You do **not** need to run `pnpm android` every day. Use:

```
# Tab 1 — backend
cd ~/Desktop/expo-crud-starter && pnpm mock:api

# Tab 2 — start the emulator first, then:
cd ~/Desktop/expo-crud-starter && pnpm start -c
#   then press  a  to open the app on Android
```

When…	Run
You only changed JS/TS code	Nothing. Fast Refresh updates the app.
You changed `.env`	Stop Metro (Ctrl + C) → `pnpm start -c`
You pulled new code with new packages	`pnpm install`
A **native** library was added, or `app.config.ts` changed	`pnpm install` → `pnpm android` (rebuild)
Open the developer menu in the app	Press `m` in the Metro terminal, or `Cmd + M` in the emulator
Reload the app	Press `r` in the Metro terminal

11. Troubleshooting

Problem / error message	Fix
`command not found: pnpm / adb / emulator / java`	Redo Part 6, then quit and reopen Terminal.
`NitroModules are not supported in Expo Go` or "*native module could not be found*"	You opened the project in **Expo Go**. Uninstall Expo Go from the emulator, then run `pnpm android` to install our development build (Part 1).
`SDK location not found. Define ANDROID_HOME…`	Redo Part 6. Check that `echo $ANDROID_HOME` prints a path that exists.
`Unsupported class file major version` / Java version errors	The wrong Java version is in use. Make sure `java -version` shows **17** (Part 6).
`No Android connected device found`	Start the emulator and check `adb devices` (Part 8). If needed, run `adb kill-server && adb start-server`.
`INSTALL_FAILED_INSUFFICIENT_STORAGE`	Device Manager → ⋮ next to the emulator → **Wipe Data**, or increase Internal Storage (Part 7).
Products tab shows **Network Error**	Check that `pnpm mock:api` is running, that `.env` uses `10.0.2.2` (emulator), then restart with `pnpm start -c`.
Red screen: "*Unable to load script*"	Metro isn't running. Run `pnpm start` and press `a`. On a real phone, run `adb reverse tcp:8081 tcp:8081`.
`Port 8081 already in use`	`lsof -ti:8081 | xargs kill -9` then try again

Gradle build fails with strange errors	Clean and rebuild:
	`cd android && ./gradlew clean && cd ..`
	`pnpm android`
	If it still fails: `rm -rf android` then `pnpm android`. The folder is regenerated.

Old screens or weird behaviour after a change	Stop Metro → `pnpm start -c` (clears the cache)
`pnpm install` fails with **401 Unauthorized**	Your Mac points to a private package registry. Run: `npm config set registry https://registry.npmjs.org/` and try again.
"*Use pnpm instead*" / `only-allow` error	You ran `npm install` or `yarn`. Use `pnpm install`.
Emulator is very slow or black screen	Use an **arm64** image on Apple Silicon, set Graphics to **Hardware**, close other heavy apps, or cold boot: `emulator -avd Pixel_8 -no-snapshot-load`
Still stuck?	Run `pnpm doctor` and share the output and the full error with the team.

12. Quick checklist

✓	Step	Check with
□	Homebrew, Node, Watchman, Git, JDK 17, pnpm installed	`node -v && pnpm -v && java -version`

☐	Android Studio setup wizard finished (Standard)	`ls ~/Library/Android/sdk`
☐	SDK Platform 36, Build-Tools 36, Platform-Tools, Emulator, Command-line Tools	SDK Manager
☐	JAVA_HOME and ANDROID_HOME set in `~/.zshrc`	`echo $ANDROID_HOME`
☐	Emulator created and booted	`adb devices` → `device`
☐	Repo cloned and `pnpm install` done	`node_modules/` exists
☐	`.env` → `http://10.0.2.2:3001/`	`cat .env`
☐	Mock API running	`pnpm mock:api`
☐	App built and opened	`pnpm android`
☐	Logged in and Products tab loads	In the app

More details in the repository: `docs/01-getting-started.md`, `docs/12-environments-and-builds.md`, `docs/14-debugging-and-common-mistakes.md`, `docs/16-mock-api.md`.